
The Forest

How a vocabulary becomes a wood — the growth mechanism, the six stages, and the technologies that draw them

Written for: someone who needs to explain or extend this feature

Scope: `app/services/forest.py`, `app/services/fsrs.py`, `features/forest/`

System: FluencyOS · Electron + React client, FastAPI + SQLite server

Scheduler: FSRS 4.5, implemented in-repo, published default weights

Scene: `three.js`, procedural low-poly geometry with optional GLB packs

1. What the forest is

One tree per saved word. The tree's size is how well the word is remembered, its colour is how much trouble it is in, and **none of it is stored**. There is no trees table, no growth counter, no per-tree row anywhere in the database.

Everything on the screen is computed, on request, from tables the rest of the app already maintains:

`vocab_words` for what was saved, `review_cards` for how well it is held, and `review_logs` for how it was used. The forest is a drawing of the scheduler's state, not a second copy of it.

That is a deliberate design choice with a practical consequence. A stored tree would drift: the forest would show a thriving oak for a word the scheduler knows was forgotten last week. Because nothing is stored, the two can never disagree, and the forest cannot be gamed. It grows when the learner's memory grows, and by no other means.

The one exception. Focus sessions are written down (`forest_focus_sessions`), because sitting through twenty-five minutes is a decision, not something derivable from other tables.

What the screen shows

- the planted clearing itself, in 3D, one plant per word;
- **Growth** — how many words stand at each of the six stages;
- **Tallest tree** — the best-held word in the collection;
- **Focus session** — a timer you commit to and mark as sat through;
- a detail card for whichever plant is hovered or clicked: its stage, how long it holds for, its health, its lapses and how often it has been said unprompted.

2. Technologies

Layer	Technology	Where
Desktop shell	Electron 33	<code>electron/main/</code> — owns the window and starts the backend
Screen	React 18 + zustand	<code>features/forest/Forest.tsx</code> , <code>store/forestStore.ts</code>
3D scene	three.js 0.186	<code>features/forest/three/</code> — scene, tree models, asset loaders
API	FastAPI	<code>routers/forest.py</code> — three routes, one of them read-only
Logic	Python, no dependencies	<code>services/forest.py</code> — stages, health, dormancy
Scheduling	FSRS 4.5, implemented in-repo	<code>services/fsrs.py</code> — ~100 lines of published arithmetic
Storage	SQLite	<code>vocab_words</code> , <code>review_cards</code> , <code>review_logs</code>

The three routes

GET <code>/forest?user_id=...</code>	the whole forest, in one query
POST <code>/forest/focus</code>	start a focus session (1-180 minutes)
POST <code>/forest/focus/{id}/complete</code>	mark it sat through, checked against the clock

The scene is loaded lazily behind a React `Suspense` boundary, so three.js and the geometry are not part of the bundle the app opens with — the forest pays for itself only when it is visited.

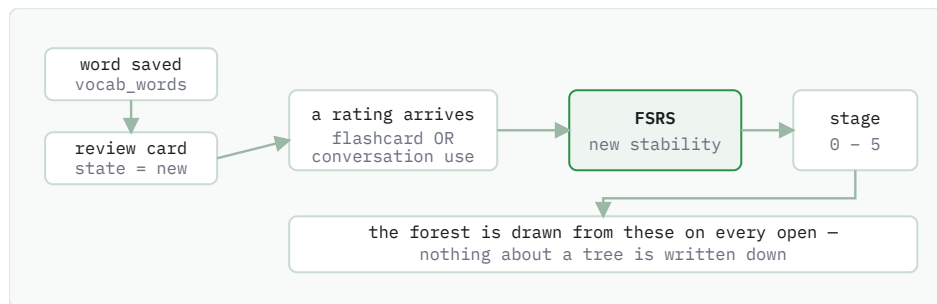
Why FSRS is implemented here rather than installed

The published `fsrs` package would be one more thing to fetch at launch. The app starts its backend with `uv run`, which re-syncs dependencies on every start, so a package that cannot be downloaded does not degrade the scheduler — it stops the whole application from starting. The algorithm is about a hundred lines of published arithmetic with no dependencies of its own, so it lives in the repository.

3. The growth mechanism

A tree's size is one number: the word's **stability**. FSRS defines stability as *the number of days until the probability of recalling this word falls to 90%*. A word with a stability of 3 is one you would probably still know in three days; a word with a stability of 90 is one you would probably still know in three months.

That number is already a measure of how well-rooted a word is, so the six stages are simply readable names for bands of it.



One word, from saved to planted

The six bands

The thresholds live in one line of `forest.py`: `STAGE_DAYS = (1, 3, 7, 21, 60)`. They climb roughly threefold because stability itself grows that way — the distance from Seed to Sprout is a day, and from Young tree to Ancient is two months.

Stage	Stability	What it means about the word
Seed	never reviewed, or < 1 day	Saved, but nothing yet shows you can hold it. Every word starts here.
Sprout	1 – 3 days	Held overnight. One successful review, or a shaky one.
Seedling	3 – 7 days	Held for most of a week.
Sapling	7 – 21 days	Held for a week or more. Past the point where most forgetting happens.
Young tree	21 – 60 days	Held for a month. This word is largely yours.
Ancient tree	60 days +	Held for two months or more, and climbing by years per review.

A new card is always a **Seed**, whatever stability it claims. `stage_for` checks `state == "new"` first, so a word that has never been answered cannot appear as anything else.

What moves a word up

Only a rating. The first one sets the stability outright, from the FSRS default weights:

First answer	Stability it sets	Stage it lands in
Again	0.49 days	Seed
Hard	1.40 days	Sprout
Good	3.71 days	Seedling
Easy	13.82 days	Sapling

After that, each successful recall multiplies stability by a growth factor that depends on how hard the card is, how stable it already was, and how close to forgetting it you were when you answered. Answering a card *late*, when recall had genuinely decayed, grows it more than answering it early.

Run with the real scheduler, always answering **Good** exactly when the card falls due:

```
review 1: stability    3.7 d   next in    4 d   -> Seedling
review 2: stability   22.7 d   next in   23 d   -> Young tree
review 3: stability  108.6 d   next in  109 d   -> Ancient tree
```

So an Ancient tree takes only three correct answers — but it takes roughly **four weeks of calendar time** to give them, because each one has to wait for the card to come due. This is the mechanism that makes the forest honest: you cannot sit down for an hour and grow one.

What brings a word down

Forgetting. A lapse does not nudge stability, it collapses it, from the same real scheduler:

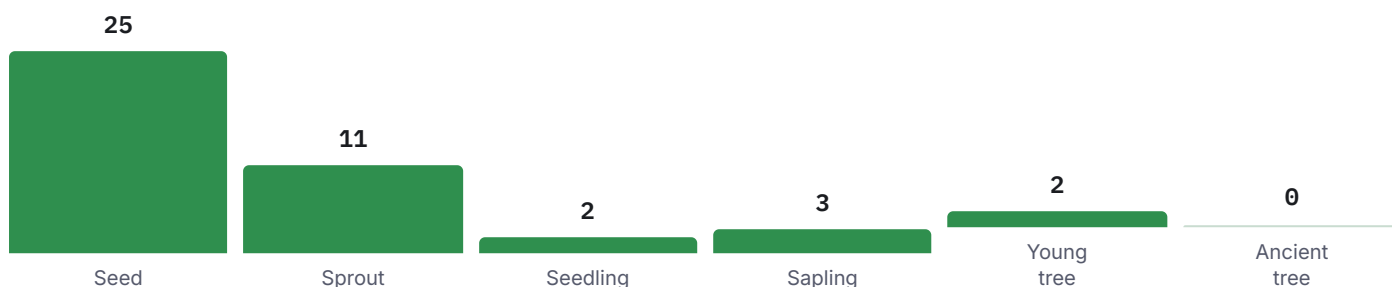
```
40.0 d   ->  4.3 d           Young tree -> Seedling
```

The tree shrinks back to where the memory actually is. It is the same number the scheduler will use to decide when you next see the card, so the picture and the queue always agree.

Ratings come from two places. A flashcard answer is one. Using the word in conversation is the other — spoken outcomes map onto the same four ratings and move the same FSRS state. A word you say unprompted grows its tree exactly as a card you answered would.

4. Reading your own forest

The Growth list is a census: how many words stand at each stage right now. A distribution of 25 / 11 / 2 / 3 / 2 / 0 — 43 words in total — says something quite specific.



What each number is telling you

Seed: 25. These are words you have saved but not yet answered — or answered and got wrong. They are not weak memories; for the most part they are memories that have not been tested at all. This is the normal shape for a collection that is being filled faster than it is being reviewed, which is what reading with lookups does. A single review session moves most of them.

Sprout: 11. Answered once, holding for a day or two. This is the most fragile band — a word here is genuinely at risk, and the scheduler will bring it back within days.

Seedling 2, Sapling 3, Young tree 2. Seven words have survived several reviews spread over weeks. The dip at Seedling is not a problem: these bands are narrow (3–7 days, then 7–21) and a word passes through them quickly when reviews are answered on time.

Ancient tree: 0. Expected, and not a failure. Sixty days of stability cannot be reached in less than about a month of elapsed time whatever you do, because each review has to wait for its due date.

The shape to aim for is a pyramid that slowly inverts. Early on, almost everything is a Seed. As reviews accumulate, the mass moves right. If the Seed count keeps climbing while the right-hand bands stay flat, the collection is growing faster than it is being reviewed — the fix is review sessions, not more saving.

Why the tallest tree matters

The panel names the best-held word in the collection. It is the one number in the forest that can only go up honestly: it is the highest stability any word has reached, and nothing but repeated, correctly-spaced recall produces it.

5. Health, wilt and dormancy

Size says how well a word is held. Colour says whether it is in trouble. The two are computed separately and deliberately do not mix.

```
health = max(0, 100 - lapses × 12)           # permanent scars
if overdue:
    wilt  = min(1, overdue_days / 14)        # current trouble
    health = health × (1 - wilt)
```

Lapses are scars: a word forgotten three times carries that history permanently, at twelve points each. Being overdue is a wilt: it deepens for a fortnight and it lifts the moment the word is reviewed. They are kept apart because one number covering both would say the same thing about a word forgotten three times and relearned — which is fine now — and a word never forgotten but three weeks overdue, which is not.

State	Canopy	Meaning
Healthy	green #63b355	No lapses, not overdue.
Wilting	towards amber #c2a85a	Lapsed before, or overdue now. Still recoverable.
Dormant	grey #9a9a8e	Suspended, or more than 30 days overdue.

A dormant plant has dropped its leaves rather than died: reviewing the word wakes it, at whatever stability it still holds. Its stage does not change while it sleeps — dormancy is about attention, not about memory.

6. How the scene draws it

The 3D clearing is built from the same numbers, with nothing added and nothing stored.

Property	Comes from	Rule
Which plant	a hash of the word	Broadleaf, conifer or shrub. The same word is always the same plant, so the forest is recognisable between visits.
How big	the stage	Scale runs 0.42 → 1.9 across the six stages.
Canopy colour	health and dormancy	Green through amber; grey when dormant.
Accent colour	a hash of the word	Petals and caps keep their own colour whatever the health, or the plot stops reading as a garden.
Where it stands	blue-noise sampling	For each plant, twelve candidate spots are thrown and the one furthest from everything already planted wins.
How far apart	the mean stage	Spacing grows with the size of what is planted, so a young vocabulary reads as undergrowth and a mature one as woodland.

Two traps this avoids, both tried first: a plain grid reads as an orchard, and jittering a grid only produces a wonky orchard because the eye finds the rows anyway. Pure random placement is the other extreme — it clumps, and trees end up standing inside each other.

The geometry is procedural: cones and a tapered cylinder with flat shading, which is how low-poly foliage is made, so the scene is complete with nothing downloaded. A converted GLB model pack drops in alongside if one is present.

7. Focus sessions

A focus session is a block of time — between 1 and 180 minutes — that the learner commits to. It is the only part of the forest that is written to the database, because it is the only part that is a decision rather than a derivation.

Completion is checked against the clock, not taken from the client: a timer marked complete by a button press is a button, not a commitment. It pays nothing. The record that it happened is the whole of it.

Previously: focus minutes and review outcomes earned *sunlight*, a currency spent on streak freezes and on reviving dormant trees. It has been removed. The streak freeze was the reason — it could be bought, it was counted on screen, and nothing in the streak calculation ever consulted it, so the streak it promised to protect broke anyway. The forest now rewards nothing and reflects everything.

8. Where to look in the code

File	What is in it
<code>app/services/forest.py</code>	<code>STAGES</code> , <code>STAGE_DAYS</code> , <code>stage_for</code> , <code>health_for</code> , the single query behind <code>trees()</code> , focus sessions
<code>app/services/fsrs.py</code>	The scheduler. Stability, difficulty, intervals, the 17 published weights
<code>app/services/review.py</code>	Where ratings arrive and cards are written; mastery levels
<code>app/routers/forest.py</code>	The three routes
<code>features/forest/Forest.tsx</code>	The screen: growth census, tallest tree, detail card, focus timer
<code>features/forest/three/ForestScene.tsx</code>	Scene, camera, planting, lighting
<code>features/forest/three/treeModels.ts</code>	Species, scale per stage, canopy and accent colour
<code>store/forestStore.ts</code>	Fetching and focus-session state
<code>backend/tests/test_forest.py</code>	17 tests: stage boundaries, health, dormancy, focus sessions

The one sentence to remember

The forest has no state of its own. Every tree is a rendering of one number the scheduler owns — how many days that word will survive in memory — so the wood can only grow when the memory does.